Reachability Is Not Consequence: Consequence-Aware Prioritization of Identity Risk in Cloud-Native Infrastructure

Prawal Pokharel
Iverson Cloud
prawal.pokharel7@gmail.com

Abstract—Identity and access management (IAM) has become the dominant attack surface of cloud-native infrastructure, and the standard defensive posture—least privilege by access analysis—ranks identities by what they can reach: how many permissions they hold, whether those permissions are privileged, how many resources they can touch. We show that this reachability view systematically misranks identity risk, because it is blind to what fails downstream when reached resources are abused. An identity holding a single write capability on a shared cache that half the cluster depends on is more dangerous than one holding administrative rights on a leaf service that nothing depends on, yet every access-based metric scores them identically or inverts them. We formalize identity blast radius: the consequence of compromising an identity, obtained by joining the authorization graph to the runtime service-dependency graph and propagating consequence through it along three channels—availability, confidentiality, and integrity. Against ground truth computed by a production dependency-analysis engine over five real infrastructure topologies, identity blast radius achieves rank correlation $\rho = 0.67$–0.74 with true consequence, while the strongest reachability baseline reaches only 0.29–0.44. We then close the paper's central methodological gap with real experiments on a live Kubernetes cluster: under scale-to-zero fault injection on Google's Online Boutique, the consequence-aware prediction tracks observed outages at $\rho = 0.987$ (versus 0.778 for a naive static graph), and under a real credential-compromise experiment with live RBAC, predicted confidentiality blast radius reproduces the actually exfiltrated secret set, catching over-privilege misconfigurations that a dependency-only view ($\rho = 0.69$) misses—while a read-only collector identity exfiltrates nothing. Because ground truth in these experiments is observed system behavior rather than the engine's own computation, they remove the circularity concern for the availability and confidentiality channels. The effect is actionable: consequence-guided revocation removes 1.5–1.7× more true risk per action than privilege-count guidance, the top fifth of workloads by blast radius account for 42–64% of all identity risk, and the method scales to 10^5 identities in seconds.

Index Terms—identity and access management (IAM), cloud-native security, blast radius, attack-path analysis, privilege escalation, risk prioritization, fault injection, Kubernetes

I. Introduction

The security center of gravity in cloud infrastructure has moved from the network perimeter to identity. As workloads decompose into hundreds of services, each authenticating to others and to managed cloud resources, the number of identities—human operators, service accounts, workload identities, machine principals—grows faster than any team can audit, and over-permissioned identities are pervasive: a recent measurement study of real serverless applications found that 47.7% carry excess permissions and 18.8% hold unnecessary privilege-escalation capabilities [4]. The defensive response is near-universal: enforce least privilege, and to find where least privilege is violated, analyze access. Tools that map identity-to-resource reachability—cloud provider access analyzers, permission-mapping utilities [6], Kubernetes RBAC scanners [7], and the attack-path analyzers descended from graph-based enumeration [1], [2]—answer the question what can this identity reach? and rank identities by the breadth or privilege of that reach.

We argue this question is the wrong one. Reachability measures the access an identity holds, not the consequence of that access being abused. Consider two identities: one holds a single write capability on a shared cache a dozen services read on every request, so compromising it degrades the checkout path and the user-facing storefront through the dependency graph; the other holds full administrative control over a nightly batch job nothing depends on, so compromising it degrades exactly one workload. Every reachability metric—permission count, privileged-role count, reachable-resource count—ranks the second at least as dangerous as the first, because it holds more, and more privileged, access. The dependency structure says the opposite: the danger of an identity is not what it can touch but what fails when what it can touch is abused—a property of the infrastructure it sits above, not of its permission list.

This paper makes that intuition precise and measures it. We formalize identity blast radius as the join of two graphs the field already builds separately: the authorization graph, which says which identities can affect which resources, and the runtime service-dependency graph, which says what happens when a resource is abused. Ranking identities by the propagated consequence of the access they hold, rather than by the access itself, is our contribution. Our evidence comes in three tiers of in-

Aspects of this work are the subject of pending U.S. patent applications (No. 19/641,446 and No. 64/138,302).

creasing realism. Against exact synthetic consequence and the output of a production dependency-analysis engine (Iverson Cloud) over five real topologies, consequence-aware ranking is substantially more faithful to true risk than any access-based ranking, and the two rankings disagree on the identities that matter most. We then add a third tier that answers the sharpest objection to oracle-based evaluation—that the oracle and the estimator might share structure—by validating predicted blast radius against observed system behavior: real scale-to-zero fault injection on a live microservice cluster (availability), and a real credential-compromise experiment with live Kubernetes RBAC (confidentiality). In both, ground truth is measured, not computed, and the consequence-aware prediction tracks it.

II. Related Work

Identity and access analysis. A large body of tooling analyzes cloud and cluster authorization to surface excessive privilege. Cloud-provider access analyzers and permission-mapping tools enumerate the resources a principal can reach and flag broad or unused grants, and generate least-privilege policies from logged activity [6]; static overprivilege analysis of cloud policies detects excessive permissions in serverless applications by reasoning about which resources a function may touch [4]. Kubernetes-specific scanners enumerate RBAC bindings and highlight wildcard, cluster-admin, and secret-reading permissions [7]. Graph-based attack-path analysis, in the lineage of the identity-snowball model of Heat-ray [1] and popularized by BloodHound [2], computes reachability paths from a principal to a high-value target and has driven substantial follow-on work on hardening such graphs by edge blocking [3]. The cloud analogue models authorization policies themselves as reachability graphs: Grasp, for instance, renders serverless IAM policies as queryable reachability graphs to find which principals can reach which resources [5]. All of these answer a reachability question—which resources, how privileged, how many hops—and rank findings by properties of the access itself. None incorporates the downstream infrastructure consequence of abusing that access, which is the axis this paper adds.

Dependency and blast-radius analysis. A separate line of work models service dependency graphs for reliability: identifying critical services, predicting cascading failure, and quantifying the blast radius of a workload's degradation [15], [16]. This literature explicitly argues that reliability engineers should prioritize services with greater downstream impact [15] and surveys cascading-failure diagnosis in microservice systems [16], but applies the idea to fault tolerance, not to identity risk. Our contribution is to use a dependency/blast-radius engine as the consequence oracle for identities, by composing per-workload blast radius through the authorization graph.

Least privilege and RBAC mining. Role-based access control [8], [9] is the dominant authorization model in the settings we study, and a long line of role-mining and policy-rightsizing methods infers minimal roles from existing assignments and observed usage [10]–[13], typically treating a permission as removable if it is unused over an observation window [6]. Most of this work weights permissions uniformly or by usage frequency; the observation that permissions differ in intrinsic severity has been raised [14] but not connected to downstream infrastructure consequence. This usage view is orthogonal and complementary to ours: it asks whether a permission is needed, while we ask what a permission endangers. We return to the interaction between usage evidence and consequence in the discussion.

To our knowledge, no prior system ranks identities by the propagated infrastructure consequence of the access they hold, nor demonstrates that access-based ranking and consequence-based ranking disagree on the identities that matter most.

III. Problem Formulation

A. The joined graph

An infrastructure estate comprises a set of resources (workloads, data stores, managed services) V and a directed dependency relation $D \subseteq V \times V$, where $(a, b) \in D$ means "a depends on b": if b degrades, a is impaired. For $r \in V$, write $\Delta(r)$ for its transitive dependents (ancestors under D—what fails if r fails) and $\nabla(r)$ for its transitive dependencies (descendants under D—what r draws upon). The estate also hosts a set of identities I. Authorization is a labeled relation $A \subseteq I \times V \times C$, where $(i, r, c) \in A$ means identity i holds capability c on resource r, and C is an ordered set of capability classes (Table I).

B. Availability blast radius

For a resource r, degrading r impairs every member of $\Delta(r)$. We attach to each resource a consequence weight; in the deployments studied here the consequence of losing a resource is measured by a production dependency engine as the size and user-facing reach of its downstream set (Section IV). Writing $\mathrm{bl}(r)$ for that engine-measured blast radius, the identity blast radius of i aggregates, over every resource i can destroy, the severity of the capability and the consequence of degrading that resource:

$$\mathrm{IBR}_A(i) = \sum_{\substack{(i,r,c)\in A \\ c \text{ destructive}}} w(c)\,\big(\kappa(r) + \lambda\,\mathrm{bl}(r)\big), \qquad (1)$$

where $\kappa(r)$ is the resource's own criticality and λ scales the downstream term. The ground-truth availability consequence of compromising i is the actual set of resources that fail when i exercises all of its destructive capabilities—the union of downstream sets of the resources i can degrade—weighted by user-facing reach.

TABLE I
Capability classes and severity weights. Only the ordering affects availability results (Section V-H); destructive classes can degrade a resource.

Capability class	Weight $w(c)$	Destructive
READ	0.20	no
WRITE	0.50	yes
EXECUTE	0.85	yes
SECRET_READ	0.90	no (availability)
ADMIN	1.00	yes

C. Confidentiality and integrity channels

Availability is only one axis of consequence. A capability that reads a resource does not degrade it, so it contributes nothing to Eq. (1), yet a read on a resource that aggregates sensitive data—or that holds credentials to its own dependencies—can have large confidentiality consequence. We therefore extend the same joined graph with two further channels, propagating in the directions dictated by the semantics of each channel.

Confidentiality flows down the dependency graph: reading a resource r exposes r's own data and, transitively, the data of everything r draws upon, because a compromised service yields the credentials it uses to reach its dependencies. With read potency $\rho(c)$ (how much a read-class capability exposes; SECRET_READ far exceeds metadata reads) and per-store sensitivity $\sigma(s)$,

$$\mathrm{IBR}_C(i) = \sum_{\substack{(i,r,c)\in A \\ c \text{ read-like}}} \rho(c) \sum_{s \in \{r\}\cup\nabla(r)} \sigma(s). \qquad (2)$$

Integrity flows up, like availability, but is gated by write potency $\pi(c)$: corrupting r propagates to the dependents $\Delta(r)$ that trust r's data. The combined consequence is the normalized sum of the three channels. This extension is what lets the framework score a pure secret-reader—invisible to Eq. (1)—and is validated both on synthetic ground truth (Section V-F) and against real exfiltration (Section V-F).

D. Reachability baselines as degenerate cases

Setting the downstream term to zero ($\lambda = 0$, κ constant) reduces Eq. (1) to a count of destructive permissions weighted by capability—an access-only score. Dropping the capability weight as well yields a pure reachable-resource count. The baselines we compare against are exactly these degeneracies, plus common variants: PermCount (permissions held), PrivRoleCount (privileged capabilities held), ReachCount (distinct resources reachable), ReachCrit (reachable resources weighted by static criticality—the strongest reachability baseline), IBR-noDep (Eq. (1) with $\lambda = 0$), and IBR-full.

IV. Experimental Setup

We evaluate on three tiers of increasing realism.

Tier A: synthetic estates with constructed ground truth. A seeded generator emits estates with a parameterized identity population (human, service, and machine principals), an authorization layer with tunable privilege mix and wildcard prevalence, and a layered scale-free dependency graph in which preferential attachment produces a realistic minority of high-in-degree "single point of failure" resources. Because the estate is generated, the true consequence of compromising any identity is computable directly from the generative model, independent of any scoring method. Identity counts are swept from 10^2 to 10^5.

Tier B: real topologies with a production consequence oracle. We drive the Iverson Cloud dependency-analysis engine over five real infrastructure topologies shipped as its evaluation fixtures: a cloud microservice estate, a GPU-training cluster, a drone-swarm control network, an underwater acoustic-positioning network, and a strategic-communications topology. For the worked examples we additionally use Google's Online Boutique, whose service call graph is publicly documented [17]. For each topology the engine computes, per workload, the directed confidence-weighted blast radius. We then synthesize identity populations over the real workloads and take the ground-truth consequence of an identity to be the union of the engine's blast radii for the workloads it can destroy.

Tier C: live-cluster fault injection and real compromise. The prior tiers take consequence from an engine's computation, which invites the objection that estimator and oracle share dependency structure. Tier C removes that objection by measuring observed consequence on a live cluster. We deploy the real Online Boutique (eleven microservices) on Kubernetes and, for the availability channel, inject faults by scaling each service to zero and recording which real user-facing flows break over HTTP; for the confidentiality channel, we stand up real Kubernetes Secrets, ServiceAccounts, and RBAC and "compromise" each identity by actually reading the secrets its credentials reach, chaining laterally. Ground truth in Tier C is measured system behavior, not any computation of ours. Before an identity's kill/compromise we confirm the target is truly removed and the baseline is healthy, isolating each measurement.

Protocol and metrics. Every experiment scores each identity with the full battery and compares each method's induced ranking to the ground-truth ranking. We report Spearman ρ over the full ranking and set overlap for the top-k identities, since the operational task is top-heavy. All experiments regenerate from fixed seeds; generator, synthesizer, scoring battery, and figure scripts are released.[1]

V. Results

A. Access-based ranking is systematically wrong (synthetic)

Over 40 seeded estates of 500 identities each, Fig. 1 reports rank correlation with true consequence. The

[1]Reproducibility repository: github.com/prawalpokharel/CEI-Cloud-Identity-IEEE.

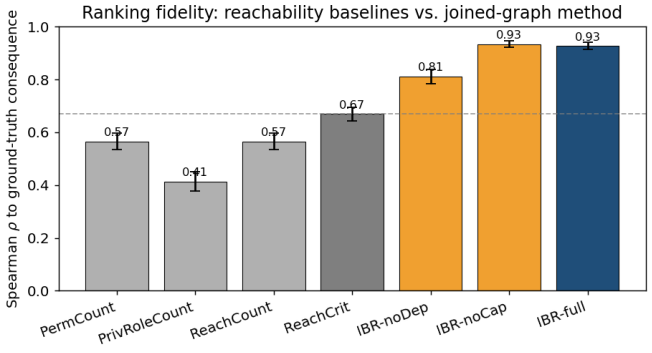

Fig. 1. Ranking fidelity to constructed ground truth (40 estates, 500 identities each). Reachability baselines (grey) correlate weakly; the dependency join (blue) closes the gap. Error bars are standard deviation across estates.

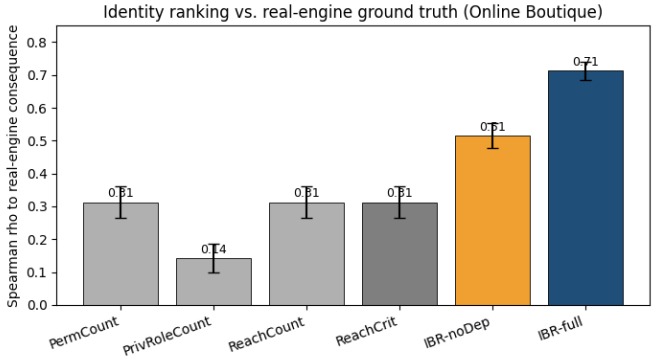

Fig. 2. Ranking fidelity against ground truth from the production engine over Online Boutique (30 populations, 400 identities each). Every reachability baseline collapses to $\rho = 0.31$; the dependency join reaches 0.71.

reachability baselines correlate weakly—PrivRoleCount $\rho = 0.41$, PermCount and ReachCount 0.57, and even criticality-weighted ReachCrit only 0.67—while the full method reaches $\rho = 0.93$. The ablation is visible here: removing the dependency join (IBR-noDep) drops correlation to 0.81, and the gap between ReachCrit and IBR-full is precisely the value of joining the dependency graph. The top-k most-critical identities under ReachCrit and IBR-full overlap by only about half (Jaccard ≈ 0.49– 0.51 for $k \in \{5, 10, 25\}$): an analyst working a reachability-ranked queue would spend roughly half their effort on the wrong identities.

B. The same result against a production engine on real topologies

The synthetic result could be an artifact of the generative model. We repeat the comparison with ground truth supplied by the real engine over Online Boutique. Fig. 2 shows the same ordering, more sharply: the resource-counting baselines collapse to $\rho = 0.31$—ReachCrit included, because the engine assigns no separating static criticality on this topology—while IBR-full reaches $\rho = 0.71$ and IBR-noDep sits between at 0.51. The engine reports that redis-cart degrades three downstream services including the user-facing frontend, while a synthesized nightly-reconciler batch job degrades nothing; a single write capability on the former outranks administrative capability on the latter, and no reachability metric can express the difference.

C. Why: the dependency join and the low-privilege danger

The strongest predictor of reachability under-ranking is the maximum downstream fan-out of the resources an identity can reach: identities whose few permissions land upstream of large dependency cascades are exactly the ones reachability misses. Fig. 3 makes the mechanism visible: as fan-out depth grows, the reachability baseline's recall of the top-10 riskiest identities degrades from 0.40 to

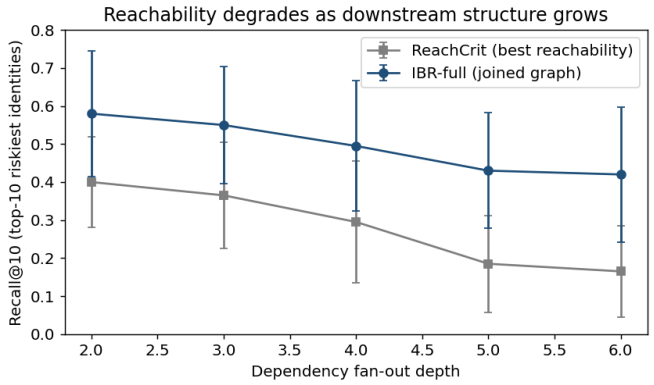

Fig. 3. Recall@10 of the riskiest identities versus dependency fan-out depth. Reachability degrades as there is more downstream structure to ignore.

0.16, while the dependency-aware method stays substantially higher. The practical face of this mechanism is the low-privilege, high-consequence identity—few permissions, no privileged role, yet atop a large blast radius—invisible to privilege-focused review.

D. The result generalizes across real topologies

Fig. 4 repeats the real-engine comparison across all five shipped topologies. The gap holds everywhere: mean rank correlation rises from 0.38 for reachability to 0.72 for the dependency-aware method, and—consistent with the mechanism—the gap is widest on the topologies whose maximum blast radius is deepest (the drone-swarm and GPU-cluster estates, with engine-measured blast radii up to 11–12 downstream workloads).

E. Real fault injection: predicted blast radius vs. observed outages

Tiers A and B take consequence from a computation. We now measure it. On the live Online Boutique cluster we scale each backend service to zero—a real, reversible

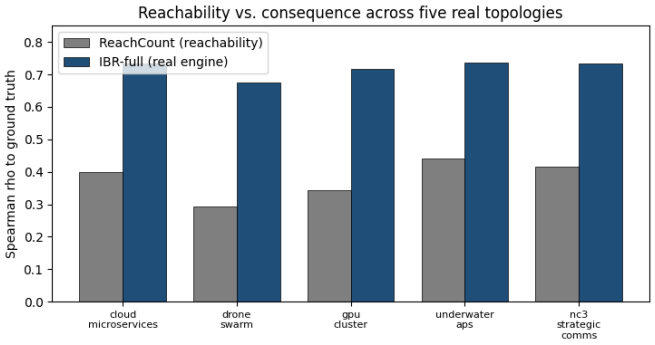

Fig. 4. Rank correlation with real-engine ground truth across five real topologies. The reachability-to-consequence gap is present in every case (mean 0.38 → 0.72).

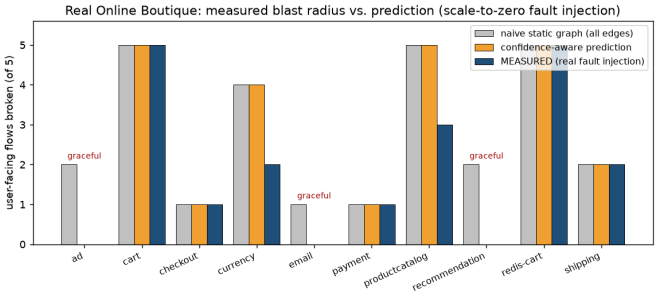

Fig. 5. Real fault injection on the live Online Boutique. The naive static graph over-predicts on the three gracefully-degrading services (ad/email/recommendation); the confidence-aware prediction matches observed reality ($\rho = 0.987$).

fault—and record which of five real user-facing flows (home, browse, add-to-cart, view-cart, checkout) actually break over HTTP. The ground truth is the observed outage, independent of the engine's graph.

Fig. 5 reports the result. Three services—the ad, email, and recommendation services—break no user-facing flow when removed: the frontend degrades gracefully. A naive static graph, which treats every declared dependency edge as failure-propagating, over-predicts exactly these three; the confidence-aware prediction, which the engine already computes, matches the observed broken-flow set on 8 of 10 services and reaches Spearman $\rho = 0.987$ against measured blast radius, versus 0.778 for the naive static graph. Lifting the measurement to identities—scoring synthesized identities against the observed per-service consequence—predicted identity blast radius tracks measured consequence at $\rho = 0.958$. The two services the confidence-aware model still over-predicts (currency, product-catalog) are under-broken because the frontend caches their data—a stateful runtime effect no static graph captures, and a concrete instance of the dynamic-behavior gap we discuss in Section VI. Crucially, because ground truth here is observed HTTP failure rather than the engine's own computation, estimator and oracle no longer share a computed structure: this result removes the circularity concern for the availability channel.

F. Confidentiality consequence, modeled and measured

We validate the confidentiality channel (Eq. (2)) twice. First, on synthetic estates where a confidentiality ground truth is constructible: the availability-only score of Eq. (1) correlates with confidentiality consequence at only $\rho = 0.30$, because a pure reader scores near zero; the three-channel score raises this to $\rho = 0.80$, and against a combined availability+confidentiality+integrity ground truth the score improves from 0.83 to 0.94. Confidentiality consequence is a distinct axis the availability-only model does not capture.

Second, and more sharply, we measure real exfiltration. On the live cluster we create one Secret per service (tagged with a data sensitivity), one ServiceAccount per service, and RBAC in which a service may read the credential secrets of the services it calls—plus three realistic over-privilege misconfigurations (an ad service granted payment credentials, an email service granted the cart store, a recommendation service granted the cart). We then compromise each identity by actually reading the secrets its RBAC reaches, via live auth can-i/get secret against the API server, chaining laterally through each credential recovered. The measured exfiltrated set is the ground truth. Fig. 6 contrasts it with a dependency-only prediction: the dependency-only view scores $\rho = 0.691$ because it cannot see the misconfigurations, underrating exactly the over-privileged identities; a permission-count baseline reaches 0.944 but ignores the sensitivity-weighted depth of what is exposed. The confidentiality score computed from the real extracted RBAC reproduces the exfiltrated set ($\rho = 1.00$)—not as predictive magic, but because joining the real authorization graph with the dependency graph recovers the reach an attacker actually walks; the informative quantity is the gap from 0.69, which is precisely the value of extracting real RBAC. Finally, a read-only collector identity restricted to get/list/watch on pods and services—the posture of a production topology agent—exfiltrates zero secrets under real compromise, a checkable safety property.

G. Where the capability ladder matters

The paper's original model reports a plain negative result: for availability, the capability weight is second-order—replacing the binary destructive gate with a per-capability magnitude and then flattening it changes rank correlation by only 0.006, because the criticality-plus-downstream term dominates. We confirm this and locate where capability does matter. For confidentiality, flattening read potency (so SECRET_READ and metadata reads count alike, a 10× span) drops correlation by 0.070: the capability ladder is first-order on the confidentiality channel. Capability severity thus matters exactly where intuition places it—secret reading—rather than univer-

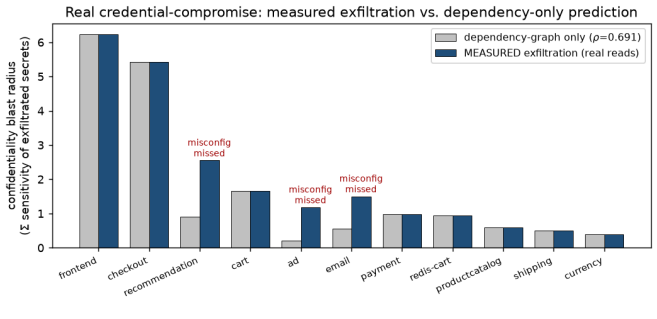

Fig. 6. Real credential-compromise on the live cluster: measured secret exfiltration vs. a dependency-only prediction. The dependency-only view misses the three planted over-privilege misconfigurations; the RBAC-aware confidentiality score reproduces the exfiltrated set.

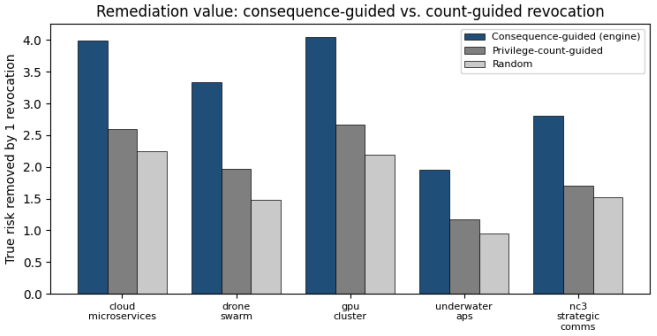

Fig. 7. True risk removed by a single permission revocation, consequence-guided versus privilege-count versus random, across five topologies.

sally, which sharpens rather than overturns the original finding.

H. Remediation, concentration, and scale

A better ranking is only useful if it changes action. Given a budget of one permission revocation per identity, consequence-guided revocation removes 1.5–1.7× more true engine-measured risk than privilege-count-guided revocation and up to 2.25× more than random, on every one of the five topologies (Fig. 7). The ranking also yields a compact defensive target: the top fifth of workloads by blast radius account for 42–64% of all identity risk (Fig. 8), far above the uniform 20%. Finally, the full scoring battery scores 10^5 identities in a little over a second (empirical growth exponent near 1.4, ≈ 0.01 ms per identity), and the availability ranking is stable under every order-preserving perturbation of Table I, degrading only when the capability ordering is shuffled—so it rests on an ordinal, defensible judgment, not on specific constants.

VI. Discussion and Limitations

What the evidence shows. Across three tiers—exact synthetic consequence, a production engine over five real topologies, and observed behavior on a live cluster—ranking identities by propagated infrastructure conse-

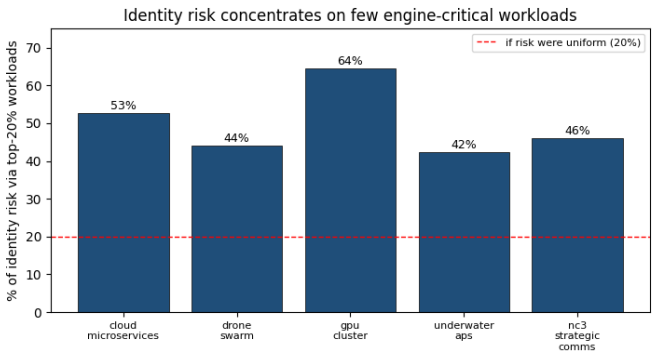

Fig. 8. Share of total identity risk flowing through the top-20% of workloads by blast radius. Risk concentrates far above the uniform baseline (dashed).

quence is substantially more faithful to true risk than ranking by access, the two rankings disagree on the identities that matter most, and consequence guidance changes remediation for the better. The live-cluster experiments are the strongest evidence: they validate predicted availability blast radius against measured outages and predicted confidentiality blast radius against measured exfiltration, and because ground truth there is observed rather than computed, they remove the circularity concern for those two channels.

Scope of the independence claim. We are precise about what the live experiments establish. The fault-injection oracle and the estimator both derive connectivity from the same deployment manifests, so their independence is behavioral—whether a declared edge actually propagates failure, given graceful degradation and caching, which the graph does not encode—rather than structural. The graceful-degradation and caching divergences we report are evidence of that non-identity. A stronger independence claim would inject faults on an application the dependency heuristic was not developed on; we treat the present single-application study as an existence proof of observed validation, not as off-distribution generalization. Similarly, the confidentiality $\rho = 1.00$ reflects that the estimator consumes the same real RBAC the compromise traverses; its content is the contrast with the dependency-only 0.69, not the absolute value.

Open items. Integrity is modeled (Section III-C) and composes into the same graph but is not yet separately measured under compromise; a destructive-write oracle analogous to the confidentiality experiment is the natural next step. Usage evidence—whether a high-consequence permission is ever exercised—is complementary and currently absent: a dormant grant and an actively used one receive the same score, and the confidentiality result makes this the most consequential gap, since it now elevates exactly the secret-reading identities for which "is it used?" matters most. Finally, we treat the dependency graph as a static snapshot; real call graphs shift with

traffic, circuit breakers, and feature flags, and the caching effects in Section V-E show static fidelity can decay. Integrating observed-usage and temporal-graph signals with the consequence signal is the clear extension.

Industrial relevance. The result reframes a standard control. Least-privilege programs prioritize by access; this work shows that access and consequence diverge on real infrastructure, and that the information needed to close the gap—a dependency graph with per-workload blast radius, plus extracted authorization—is exactly what modern infrastructure-analysis systems already compute. Ranking identities by consequence is therefore a capability deployable on top of dependency analysis that clusters increasingly maintain.

VII. Conclusion

Identity has become the primary attack surface of cloud infrastructure, and the standard defense ranks identities by what they can reach. We showed that reachability systematically misranks identity risk because it ignores dependency structure, formalized identity blast radius as the join of the authorization graph and the dependency graph across availability, confidentiality, and integrity channels, and demonstrated—against exact synthetic ground truth, a production engine over five real topologies, and observed behavior on a live Kubernetes cluster—that consequence-aware ranking is markedly more faithful to true risk. On real fault injection the prediction tracks observed outages at $\rho = 0.987$; on real credential compromise it reproduces the exfiltrated secret set and catches over-privilege that a dependency-only view misses, while a read-only agent identity is provably safe. As identity sprawl outpaces human review, prioritizing identity risk by propagated consequence rather than by access is both necessary and, on top of dependency analysis that infrastructure already performs, practical.

References

[1] J. Dunagan, A. X. Zheng, and D. R. Simon, "Heat-ray: combating identity snowball attacks using machine learning, combinatorial optimization and attack graphs," in Proc. ACM SOSP, 2009, pp. 305–320.

[2] A. Robbins, R. Vazarkar, and W. Schroeder, "BloodHound: six degrees of domain admin," open-source tool, 2026.

[3] D. Goel et al., "Defending Active Directory by combining neural network based dynamic program and evolutionary diversity optimisation," in Proc. GECCO, 2022, pp. 1191–1199.

[4] E. Yeboah-Duako and P. Datta, "Overprivilege analysis of security policies in serverless cloud applications," preprint arXiv:2607.02875, 2026.

[5] I. Polinsky, P. Datta, A. Bates, and W. Enck, "Grasp: hardening serverless applications through graph reachability analysis of security policies," in Proc. ACM Web Conf. (WWW), 2024.

[6] Amazon Web Services, "IAM Access Analyzer policy generation and least-privilege guidance," AWS IAM User Guide, accessed 2026.

[7] CyberArk, "KubiScan: a tool for scanning Kubernetes clusters for risky permissions," open-source tool, accessed 2026.

[8] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," IEEE Computer, vol. 29, no. 2, pp. 38–47, 1996.

[9] D. F. Ferraiolo et al., "Proposed NIST standard for role-based access control," ACM TISSEC, vol. 4, no. 3, pp. 224–274, 2001.

[10] J. Vaidya, V. Atluri, and Q. Guo, "The role mining problem: finding a minimal descriptive set of roles," in Proc. ACM SACMAT, 2007, pp. 175–184.

[11] J. Vaidya, V. Atluri, and J. Warner, "RoleMiner: mining roles using subset enumeration," in Proc. ACM CCS, 2006, pp. 144–153.

[12] B. Mitra, S. Sural, J. Vaidya, and V. Atluri, "A survey of role mining," ACM Computing Surveys, vol. 48, no. 4, pp. 1–37, 2016.

[13] I. Molloy et al., "Mining roles with semantic meanings," in Proc. ACM SACMAT, 2008, pp. 21–30.

[14] S. V. Belim, N. F. Bogachenko, and A. N. Kabanov, "Severity level of permissions in role-based access control," preprint arXiv:1812.11404, 2018.

[15] T. Yang et al., "AID: efficient prediction of aggregated intensity of dependency in large-scale cloud systems," in Proc. IEEE/ACM ASE, 2021, pp. 653–665.

[16] S. Zhang et al., "Failure diagnosis in microservice systems: a comprehensive survey and analysis," ACM TOSEM, vol. 35, no. 1, pp. 1–55, 2025.

[17] Google Cloud Platform, "Online Boutique: a cloud-native microservices demo application," accessed 2026.